

**NANYANG
TECHNOLOGICAL
UNIVERSITY**
SINGAPORE

Semester Group Project

by

Name	Matric No.	Work %
Timothy Chang	<i>U2220136J</i>	50.0%
Neo Zhi Xuan	<i>U2222293E</i>	50.0%

SC4023 Big Data Management

NANYANG TECHNOLOGICAL UNIVERSITY

April 2026

Abstract. We present our highly performant columnar store query engine. It implements **15 distinct optimisations**, spanning five architectural layers. They include encoding (Dictionary, Precomputed aggregates), Physical layout (Pre-sorted clustering, Per-column binary files, Town partitioning, RLE town skipping), Indexing (Zone maps, Bitmap indices, Binary search), Scan-path rewriting (Result reuse, Late materialisation, Predicate reordering, Integer gating), and I/O management (Chunked streaming, Memory-mapped I/O). All optimisations are toggleable by boolean flags. This enables us to build a combinatorial evaluation suite testing across 40 permutations of optimisations, each with correctness verification. Physical town partitioning together with Result reuse achieves a **3,389× query speedup** at only 660 KB memory. Memory-mapped I/O with Result Reuse achieves the fastest end-to-end total time of **71.9 ms**, a 67.3× total-time speedup vs. baseline (59.9× query speedup). Negative results are instructive as well. Bitmap indexing (0.15×) and Predicate reordering (0.06×) show that removing cheap predicates via an expensive process greatly affects performance, especially when data is already well filtered upstream.

1 Data Storage

1.1 System Architecture

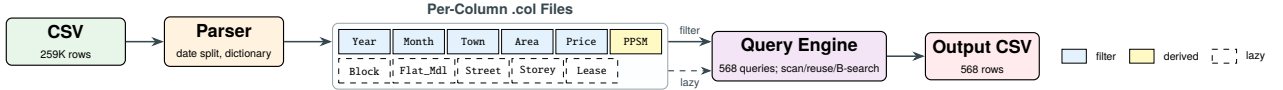


Figure 1: Pipeline: CSV → parser → per-column binary files → engine → output CSV.

Our parser detects column positions from the CSV header and streams rows into parallel `std::vector` columns. YYYY-MM date strings are split into Year (uint16) and Month (uint8). Invalid rows (malformed dates, empty fields, etc) are skipped.

1.2 Base Design of Column Store

The ColumnStore struct holds one `std::vector` per column. Numeric columns use fixed-width types as seen in Table 1. All columns are populated in one pass through the CSV. For the baseline, when we load all 10 columns into memory, it incurs a total of 71.4 MB in memory.

1.3 Per-Column Physical Layout

Column	Type	Encoding	B/row	Derived?	Rationale
Year	uint16	raw	2	from YYYY-MM	range predicate
Month	uint8	raw	1	from YYYY-MM	range predicate
Town	uint16	dict (A1)	1	no	26 towns ≤ 5 bits
Floor_Area	uint16	raw	2	no	≥ y predicate
Resale_Price	uint32	raw	4	no	aggregate target
PricePerSqm (A4)	double	raw	4	yes	removes per-row division
Block, Flat_Model, Lease_Commence	...	late-materialised (\$2.5)			output only

Table 1: Physical layout per column.

1.4 Encoding and Layout Decisions

Dictionary Encoding. Our DictionaryEncoder is a bidirectional `unordered_map/vector` pair that maps each of the unique 26 towns to a `uint16_t` ID. During queries, our engine would first resolve the target town list to IDs once outside the scan, before checking for integer equality in the inner loop. This brought about a 32% query speedup (2,299→1,573 ms) at the cost of +2.9% memory and 16.7% slower load time. The increased load time is a one-time penalty amortised across all the queries.

Column Splitting of Month. We split the date string into two integer columns at parse time. This eliminates repeated string parsings and allow for direct `uint16` range predicates. Using just the Year predicate alone eliminates ~90% of rows.

Precomputed PricePerSqm. The storing of (price / area) as double will reduce the per-row floating point division we had to do for entries that survived the filter. Only ~0.2% of row iterations reach this division, the ~543 survivors per query. This is why the measured speed up is only <0.1%.

Pre-sorted Clustering. A `stable_sort` by the key (Town, Year, Month) will physically reorder all columns once at ingestion. A linear pass then records each town partition’s [begin, end) boundaries at a $O(1)$ lookup cost at query time. This is a building block for binary search (§2.4) and zone map chunking (§1.5).

Separate Binary Column Files. Every column is written to its own binary file during the CSV to columnar database conversion. During query time, only the five filtered columns are loaded. Output columns are lazy loaded for each surviving row per (x, y) pair. The value of this optimisation is in memory and load-time reduction; memory usage drops by 96.2% from 71.4 MB to 2.7 MB, while load time also significantly falls from 2,536 ms to 68 ms. However, it incurs a 2.0% slowdown in query time, because the binary I/O path still performs a full scan.

1.5 Zone Maps

Every numeric filter column is divided into fixed size chunks of 1,024 rows, each storing a (min, max) entry. Hence, there will be 254 chunks in total, with a size of <8 KB. With §1.4, a significant 96.1% of chunks are skipped across all 568 queries (138.6K of 144.3K chunks skipped), proving its time efficiency.

1.6 Bitmap Index on Town

We build one `std::vector<bool>` for each distinct town at load time (26 bit-vectors $\times$ 259K bits $\approx$ 842 KB). Before querying, bit vectors for the specific towns are OR combined into a `uint8_t`-per-row mask ($\sim$ 259 KB). This replaces the per-row town predicate with a single array lookup. The mask uses one byte per row for cache line friendly sequential access. As explored in §3.5, this optimisation hurts performance. Eliminating all 12.55M Town comparisons yielded just 0.15 $\times$ throughput. Its overhead outweighed the cost of original comparisons as the Year predicate already filters out $\sim$ 90% of rows before Town is reached.

1.7 I/O Layer: Chunked and Memory-Mapped

Chunked I/O. Our system streams filtered columns from §1.4 in bounded size chunks under a configurable memory budget. The outer loop iterates over the I/O chunks, while the inner loop runs all 568 queries per chunk. It exploits the associativity of min to merge per-chunk winners.

Memory-mapped I/O. This feature replaces `ifstream/fread` with POSIX `mmap(2)`. It means each `.col` file is mapped into virtual address space with a single syscall, then `memcpy`'d into the existing column vectors. Load times drop from 68 ms (A9 `ifstream`) to 31 ms, a 55% reduction. For lazy materialisation, `loadStringAtMmap` replaces `std::ifstream` construction with raw POSIX calls. Combined with §2.3, it achieves the fastest total time of **71.9 ms** (67.3 $\times$ end-to-end).

1.8 Physical Partitioning by Town

Columnar binary files are split into one subdirectory per town. This is how we implemented partition pruning. During a query, `loadColumnFilesPartitioned` opens only the subdirectories for the target town and concatenates their columns into the in-memory `ColumnStore`:

Listing 1: Simplified code for partition loading.

```
1 void loadColumnFilesPartitioned(const string& base_dir,  
2     const vector<string>& target_towns, ColumnStore& db) {  
3     for (const auto& town : target_towns) {  
4         string dir = base_dir + "/" + town;  
5         // append filter + materialisation columns from dir  
6         appendNumericCol(db.col_month_year, dir + "/month_year.col");  
7         appendNumericCol(db.col_floor_area, dir + "/floor_area.col");  
8         // ... remaining columns ...  
9     }  
10    // rebuild zone maps / bitmap indices over concatenated data  
11 }
```

For 3 target towns, only 42,935 of 259,237 rows are loaded, a 83.4% reduction. If a row is in memory, it implicitly means that it must belong to a target town. No other town comparisons are then needed. Memory footprint also drops to **660 KB**, which is 99.1% less than baseline.

1.9 RLE on Sorted Towns

We implement run-length encoding on sorted town columns by compressing `town_encoded.col` into (value, start, length) triples:

$$\text{run_value}[k], \text{run_start}[k], \text{run_length}[k].$$

The query engine resolves requested towns to run IDs and scans only the selected intervals, skipping non-matching runs entirely instead of checking town equality for every row. Without pre-sorting (§1.4), the unsorted column produces 2,570 runs; the engine still skips 122.86M rows via run boundaries, reducing rows scanned post-RLE to 24.39M and achieving a 10.5 $\times$ query speedup. With pre-sorting, each town now forms a single contiguous run (26 runs total), and town comparisons drop to zero. The strongest composition happens when including binary search techniques, which narrows each town partition to the target month range, and RLE eliminates all remaining town equality checks, yielding a 180 $\times$ query speedup (13.7 ms query time).

1.10 Exception Handling

In empty result sets, we omit that (x, y) pair from the output CSV. If there are ties in minimum price-per-sqm, the system only keeps the first one it sees during scans.

1.11 Space–Time Trade-off Summary

Decision	$\Delta\text{Mem} / \Delta\text{Load}$	ΔQuery	Verdict
Dictionary Encoding	+2.9% mem, +16.7% load	−32% query	keep
Precompute PricePerSqm	+2.8% mem	±0% query	keep (negligible)
Pre-sorted Clustering	+16.6% load	enables B3	keep (foundation)
Separate Binary Column Files	−96.2% mem, −97.3% load	+2.0% query	keep (I/O win)
Zone Maps	≈ 0% mem	−25% query	keep
Bitmap Index on Town	+1.1% mem	−582% query	reject
Memory-mapped I/O	≈ 0% mem	−55% load	keep (I/O)
Physical Partitioning by Town	−99.1% mem	−90.9% query	keep
RLE on Sorted Towns	≈ 0% mem	−90.5% query	keep (with A2)

Table 2: Storage decisions as explicit trade-offs. All deltas vs. baseline.

2 Data Processing

2.1 Baseline Scan

The baseline scans all 259,237 rows for each of 568 (x, y) pairs. Predicates are evaluated in the order Year $\rightarrow$ Month $\rightarrow$ Town $\rightarrow$ Area; Year is checked first because it eliminates $\sim 90\%$ of rows with a single uint16 equality before the more expensive Town iteration runs.

Listing 2: Baseline scan loop in runQuery (simplified).

```

1 for (size_t i = 0; i < N; ++i) {
2     if (db.col_month_year[i] != target_year) continue; // ~90% eliminated
3     if (db.col_month_month[i] < start_month ||
4         db.col_month_month[i] > end_month) continue;
5     // Town: linear scan over 7-element target list
6     bool match = false;
7     for (const auto& t : towns)
8         if (db.col_town[i] == t) { match = true; break; }
9     if (!match) continue;
10    if (db.col_floor_area[i] < y) continue;
11    double ppsm = (double)db.col_resale_price[i] / db.col_floor_area[i];
12    if (ppsm < min_ppsm) { min_ppsm = ppsm; best_i = i; }
13 }
```

Across the full grid this produces 147.25M row iterations and 12.55M town comparisons, averaging 4,835.6 ms total (2,536.1 ms load + 2,299.5 ms query).

2.2 Composable Scan Architecture

All 15 optimisations are controlled by independent boolean flags in the ColumnStore struct (toggled via OptConfig in the evaluation suite). The query engine has three paths: (1) a *reuse path* (§2.3) that short-circuits via the cumulative table, (2) a *unified scan loop* where each flag controls one orthogonal decision; **Dictionary Encoding** selects int vs. string comparison, **Zone Maps** wraps the loop in chunk-level pruning, **Bitmap Indexes** replaces the town predicate with a bitmap mask, **RLE** replaces per-row town checks with run-level skipping, **Predicate Ordering** reorders predicates, **Integer Early-Exit Gates** float computation, **Precomputed PricePerSqm** selects the PPSM source, and **Late Materialization** defers the price column to a Phase 2 survivors pass, and (3) a *partition-pruned path* (§1.8) that loads only target-town data before entering either path (1) or (2). **Memory-mapped I/O** is an I/O-layer substitution under **Separate Binary Column Files**, transparent to the scan logic. This composability allows the evaluation suite to test ~ 40 configurations and isolate each contribution.

2.3 Result Reuse

The 568 queries overlap: for fixed y , the month window for $x=k$ is a superset of $x=k-1$; for fixed x , every record satisfying $\text{area} \geq y$ also satisfies $\text{area} \geq (y-1)$, so minima propagate from higher to lower thresholds.

Three-step Cumulative Table Build. Step 1 (single $O(N)$ scan): For each row passing year and town filters, compute $\text{offset} = \text{month} - \text{start} + 1$ (range 1–8) and $\text{bucket} = \min(\text{area}, 150)$. Records with $\text{area} > 150$ are *clamped* to bucket 150, not dropped. This is correct because every valid y threshold is ≤ 150 , so a record with area 160 satisfies every y and must contribute to all buckets it qualifies for. The clamped bucket propagates into all lower buckets during the Y-sweep. **Step 2 (X-sweep):** For each area bucket, sweep offsets $1 \rightarrow 8$, propagating a running minimum. **Step 3 (Y-sweep):** For each offset, sweep area $149 \rightarrow 80$, propagating the minimum downward. After Step 3, $\text{cum_table}[x][y]$ holds the answer for each (x, y) pair, and the query phase becomes a single $O(1)$ array lookup. The full C++ implementation is given in Figure 7 (Appendix).

Results. This optimisation alone reduces query time from 2,299.5 ms to 4.6 ms (499 $\times$ query speedup), with rows scanned dropping from 147.25M to 259.2K (the single build scan). When combined with **Dictionary Encoding**, query time drops further to 3.1 ms (749 $\times$).

2.4 Pre-sort + Binary Search

With data pre-sorted by (Town, Year, Month), the query engine runs custom `lowerBound/upperBound` searches within each town partition using a linearised month key ($\text{year} \times 12 + \text{month} - 1$):

Listing 3: Binary search on linearised month key.

```
1 inline uint32_t monthKey(uint16_t year, uint8_t month) {
2     return (uint32_t)year * 12u + (uint32_t)(month - 1u);
3 }
4 size_t lowerBoundMonthKey(const ColumnStore& db, size_t l, size_t r,
5     uint32_t key) {
6     while (l < r) {
7         size_t mid = l + (r - l) / 2;
8         if (monthKeyAt(db, mid) < key) l = mid + 1; else r = mid;
9     }
10    return l;
11 }
```

Rows scanned drop from 147.25M to 751.7K (99.5% reduction), town comparisons fall to zero, and query time drops to 11.9 ms (193× query speedup). This is an independent scan-elimination path from **Result Reuse**.

2.5 Late Materialisation (Two Phases)

Phase 1 applies only the four predicate columns, appending survivor indices to a vector. Phase 2 iterates survivors to fetch `Resale_Price`, compute `PricePerSqm`, and track the minimum. Measured result: 0.7% *slowdown* (2,315 ms vs. 2,299 ms), essentially within measurement noise, because the overhead of maintaining the survivor vector roughly cancels the cache savings from deferring a single 4-byte column.

2.6 Other Scan Optimisations

Zone-map Chunk Skipping (§1.5). Before scanning a 1,024-row chunk, the engine tests its min/max against Year, Month, and Floor_Area predicates; failing chunks are skipped entirely (96.1% skip rate, 25% standalone query speedup; combined with Dictionary Encoding: 58%).

Bitmap-driven Row Filtering (§1.6). A precomputed `uint8_t` mask replaces the per-row town loop with a single byte lookup. Town comparisons drop to zero, but query time regressed to 0.15× (15,670 ms vs. 2,299 ms baseline): the byte-load-plus-branch on every row is more expensive than the original integer comparison that runs only on the ~10% of rows surviving the Year filter. This confirms that the baseline Year→Month→Town order is already near-optimal.

Predicate Reordering. Reordering to Town → Year → Month → Area caused a 0.06× regression: town comparisons surged from 12.55M to 417.51M because every row now pays the expensive Town list iteration before the cheap Year check can eliminate it. The lesson: optimal order must minimise *cost × pass_rate*, not maximise selectivity alone.

Integer Early-exit Gate. Tests $\text{price} \times \text{cur_min_area} \geq \text{cur_min_price} \times \text{area}$ in `uint64` before computing float `PricePerSqm`. Measured: within noise—too few survivors (~543/query) for the eliminated float work to register.

2.7 Correctness Argument

Each optimisation preserves output equivalence: **Dictionary Encoding** is a bijection. **Pre-sorted Clustering** is a permutation. **Precomputed PricePerSqm** is identical to runtime division. **Separate Binary Column Files** reproduce identical column vectors. **Zone Maps** skip a chunk only when its (min, max) proves no row can match. **Bitmap Index on Town** mask is the OR of exact per-town bit-vectors; the mask lookup is equivalent to the membership test it replaces. **Binary Search** locates exact boundaries in sorted data. **Result Reuse** is exact because min is associative over disjoint month ranges and monotone over area thresholds; clamping $\text{area} > 150$ to bucket 150 is correct because all valid $y \leq 150$. **Late Materialization** defers column access without altering the predicate set. **Predicate Ordering** changes evaluation order, not the predicates. **Integer Early-Exit Gate** is a sufficient condition for “cannot beat current minimum.” **Chunked I/O** processes the same rows in bounded batches; the cumulative table is merged across chunks via the associativity of min. **Memory-mapped I/O** memcpy from mapped region produces byte-identical vectors as `fread`. **RLE on Sorted Towns** run boundaries mark contiguous blocks of identical town values; skipping a non-matching run is equivalent to skipping each row individually. **Physical Partitioning by Town** partition loading concatenates disjoint town subsets; the union equals the set of rows matching the town predicate. All ~46 configurations produced byte-identical output, verified by the evaluation suite’s element-by-element output comparison (§3.4).

3 Experiment Result

Our query parameters are derived from matrix number U2220136J: target year 2016, start month 3 (March), threshold ≤ 4725 , towns {CLEMENTI, BEDOK, BUKIT PANJANG, CHOA CHU KANG, PASIR RIS}, with $x \in [1, 8]$ and $y \in [80, 150]$ yielding 568 (x, y) query pairs.

3.1 Compulsory Artefacts

```
(base) MacBook-Air:Project timchang$ make run
g++ -std=c++17 -Wall -Wextra -Iinclude -c -o main.o main.cpp
g++ -std=c++17 -Wall -Wextra -Iinclude -c -o src/column_store.o src/column_store.cpp
g++ -std=c++17 -Wall -Wextra -Iinclude -c -o src/csv_parser.o src/csv_parser.cpp
g++ -std=c++17 -Wall -Wextra -Iinclude -c -o src/query_engine.o src/query_engine.cpp
g++ -std=c++17 -Wall -Wextra -Iinclude -c -o src/output_writer.o src/output_writer.cpp
g++ -std=c++17 -Wall -Wextra -Iinclude -c -o src/column_file_io.o src/column_file_io.cpp
g++ -std=c++17 -Wall -Wextra -Iinclude -o column_store main.o src/column_store.o src/csv_parser.o src/query_engine.o src/output_writer.o src/column_file_io.o
Build successful: column_store
./column_store U2220136J
Matriculation number : U2220136J
```

(a) Compilation.

```
Target year : 2016
Start month : 3
Target towns : CLEMENTI, BEDOK, BUKIT PANJANG, CHOA CHU KANG, PASIR RIS
Detected 10 columns in CSV header.
Column positions: month=0 town=1 area=6 price=9

Data Ingestion Complete:
File : ../data/ResalePricesSingapore.csv
Lines read : 259237 (excl. header)
Records loaded : 259237
Records skipped: 0

Total records in column store: 259237
Output written to : ScanResult_U2220136J.csv
Valid (x,y) pairs : 568
Done.
```

(b) Execution (259,237 rows, 568 queries).

```
(base) MacBook-Air:Project timchang$ head -20 ScanResult_U2220136J.csv
(x, y),Year,Month,Town,Block,Floor_Area,Flat_Model,Lease_Commence_Date,Price_Per_Square_Meter
(1, 80),2016,03,CHOA CHU KANG,701,109,Model A,1995,3028
(1, 81),2016,03,CHOA CHU KANG,701,109,Model A,1995,3028
(1, 82),2016,03,CHOA CHU KANG,701,109,Model A,1995,3028
(1, 83),2016,03,CHOA CHU KANG,701,109,Model A,1995,3028
(1, 84),2016,03,CHOA CHU KANG,701,109,Model A,1995,3028
(1, 85),2016,03,CHOA CHU KANG,701,109,Model A,1995,3028
(1, 86),2016,03,CHOA CHU KANG,701,109,Model A,1995,3028
(1, 87),2016,03,CHOA CHU KANG,701,109,Model A,1995,3028
(1, 88),2016,03,CHOA CHU KANG,701,109,Model A,1995,3028
(1, 89),2016,03,CHOA CHU KANG,701,109,Model A,1995,3028
(1, 90),2016,03,CHOA CHU KANG,701,109,Model A,1995,3028
(1, 91),2016,03,CHOA CHU KANG,701,109,Model A,1995,3028
```

(c) ScanResult_U2220136J.csv.

Figure 2: End-to-end execution: build (a), ingestion + query (b), output CSV (c).

3.2 Excel Cross-Validation

We wrote an independent Office Script to verify engine outputs. The script reads the raw CSV dataset, applies the same four predicates ($\text{Year} = 2016$, $\text{Month} \in [\text{start}, \text{start} + x - 1]$, $\text{Town} \in \{\text{CLEMENTI}, \text{BEDOK}, \text{BUKIT PANJANG}, \text{CHOA CHU KANG}, \text{PASIR RIS}\}$, $\text{Floor_Area} \geq y$), and computes $\min(\text{Resale_Price}/\text{Floor_Area})$ rounded to the nearest integer; identical logic to the engine’s `runQuery`. Table 3 confirms exact agreement across six (x, y) pairs spanning different selectivities; the Appendix (Figure 5) shows the script environment and output alongside the engine’s CSV.

x	y	Engine PPSM	Excel PPSM	Match
1	80	3028	3028	✓
3	90	2878	2878	✓
4	100	2788	2788	✓
6	120	2878	2878	✓
7	130	2939	2939	✓
8	150	3387	3387	✓

Table 3: Excel cross-validation for selected (x, y) pairs.

3.3 Benchmark Table

Each configuration was benchmarked; memory is estimated from column vector capacities plus dictionary/zone-map overhead. Table 4 shows a cumulative build-up revealing three independent optimisation paths: **Result Reuse**, **Pre-sort + Binary Search**, and **Physical Partitioning by Town**.

Configuration	Load (ms)	Query (ms)	Total (ms)	Q.Speedup	Rows Scan.	Memory
Baseline	2,536.1	2,299.5	4,835.6	1.0×	147.25M	71.4 MB
+ Dict Encoding	2,958.5	1,572.7	4,531.2	1.5×	147.25M	73.4 MB
+ Result Reuse (alone)	2,518.7	4.6	2,523.3	499×	259.2K	71.4 MB
+ Dict Encoding+Result Reuse	2,920.5	3.1	2,923.6	749×	259.2K	73.4 MB
+ Separate Bin. Col. File+Result Reuse	78.5	46.6	125.1	49.4×	259.2K	2.7 MB
+ Sep. Bin. Col. File+Everything	25.0	45.7	70.7	50.3×	259.2K	5.3 MB
<i>Memory-mapped I/O path:</i>						
Memory-mapped I/O+Reuse	33.5	38.4	71.9	59.9×	259.2K	5.3 MB
Memory-mapped I/O+Everything	41.0	45.5	86.5	50.5×	259.2K	6.1 MB
<i>Alternative scan-elimination path:</i>						
Pre-sort+BSearch	3,293.5	11.9	3,305.4	193×	751.7K	71.0 MB
Sep. Bin. Col. File+Sort+BSearch	115.3	69.3	184.5	33.2×	751.7K	2.7 MB
<i>RLE town skipping path:</i>						
RLE	2,526.0	234.7	2,760.7	10.5×	24.4M	71.5 MB
Presort+RLE	2,981.1	224.7	3,205.8	10.9×	24.4M	71.0 MB
BSearch+Presort+RLE	2,979.8	13.7	2,993.5	180×	751.7K	71.0 MB
Sep. Bin. Col. File+Sort+RLE+BSearch	104.5	55.8	160.3	44.0×	751.7K	2.7 MB
<i>Partition-pruned path:</i>						
Town Partition	72.0	208.4	280.4	11.0×	24.4M	660 KB
Town Partition+ZoneMaps	74.1	168.0	242.1	13.7×	24.4M	1.2 MB
Town Partition+Reuse	72.4	0.7	73.1	3,389×	42.9K	660 KB
Town Partition+Everything	73.5	0.7	74.2	3,403×	42.9K	1.2 MB

Table 4: Benchmark across key configurations. Q.Speedup = baseline query time / config query time.

Figure 3: Speedup vs. baseline (= 1.0×) across key configurations.

3.4 Correctness Invariant

The evaluation suite (`eval_suite.cpp`) compares each configuration’s 568-entry output vector against the baseline element-by-element, checking that the (x, y) pair, all output fields, and the computed PricePerSqm are identical. All ~46 configurations report PASS.

3.5 Negative and Neutral Results

To justify trade-offs, we analysed results that were sub-optimal to understand the factors that impact efficiency. Table 5 isolates five such optimisations. **Predicate Ordering** and **Bitmap Index on Town** are the most instructive: both are textbook-recommended techniques that regressed dramatically because they violated `cost×pass_rate` ordering.

Configuration	Query (ms)	Δ vs Baseline	Verdict
Predicate Ordering	37,584.1	0.06×	Rejected
Bitmap Index on Town	15,669.9	0.15×	Rejected
Late Materialization	2,315.5	0.99×	Within noise
Precomputed PricePerSqm	2,295.9	1.00×	Within noise
Integer Early-Exit Gate	2,298.4	1.00×	Within noise

Table 5: Negative and neutral results (individual isolation).

Bitmap Index on Town is instructive alongside **Predicate Ordering**: both attempt to eliminate town comparisons, yet **Bitmap Index on Town** *regressed* because the per-row byte-load adds branch overhead on all 147M row-visits, while **Pre-sort + Binary Search** avoids visiting non-target rows entirely. **Late Materialization** produced a neutral result because deferring a 4-byte column on a narrow schema yields cache savings that roughly cancel against the overhead of maintaining a survivor index vector.

3.6 Sensitivity Analysis: Memory Budget of Memory-mapped I/O Path

Table 6 varies the chunked-I/O memory budget across a 50× range. All **Chunked I/O** configurations use **Separate Binary Column Files** with **Dictionary Encoding**, **Zone Maps**, and **Result Reuse** built into the batch runner.

Budget	I/O Chunks	Query (ms)	Total (ms)	Q.Speedup
50 MB	1	372.6	405.2	6.17×
10 MB	1	371.6	395.9	6.19×
5 MB	2	373.9	397.7	6.15×
2 MB	3	375.1	398.7	6.13×
1 MB	6	401.6	425.4	5.73×

Table 6: Chunked-I/O sensitivity. Total bytes read = 4.7 MB at all budgets.

Performance is stable from 50 MB down to 2 MB; the 7% degradation at 1 MB confirms chunked I/O adds negligible overhead at this data scale.

3.7 Environment

All benchmarks were run on an Apple M-series MacBook Air, compiled with `clang++ -O2 -std=c++17`. The dataset contains 259,237 rows across 10 columns; all 568 (x, y) query pairs were evaluated per configuration.

4 Conclusion

The largest query-time win was **Result Reuse**: a single $O(N)$ build pass replaced $568 \times O(N)$ scans with $O(1)$ lookups (499× query speedup; 749× with Dictionary Encoding). Physical Town Partitioning reduced the build-pass input from 259.2K to 42.9K rows (83.4%), so Result Reuse’s single $O(N)$ scan operated over 6× fewer rows; because these optimisations target orthogonal bottlenecks (row pruning vs. repeated scans), their gains compound to 3,389× at 660 KB. The fastest end-to-end configuration was **Memory-mapped I/O + Result Reuse** at 71.9 ms total (67.3× total-time speedup); once query time is near-zero, load time dominates, so mmap’s 55% load reduction has outsized impact. The most instructive failures were **Predicate Reordering** (0.06×) and **Bitmap Index on Town** (0.15×): **Predicate Reordering** demonstrated that optimal predicate order must minimise `cost × pass_rate`, while **Bitmap Index on Town** showed that eliminating a cheap predicate via a more expensive mechanism can regress performance when upstream predicates already prune effectively. Across all 15 optimisations and ~40 tested configurations, every result was byte-identical to the baseline; correctness was never sacrificed for speed.

Appendix: Figures

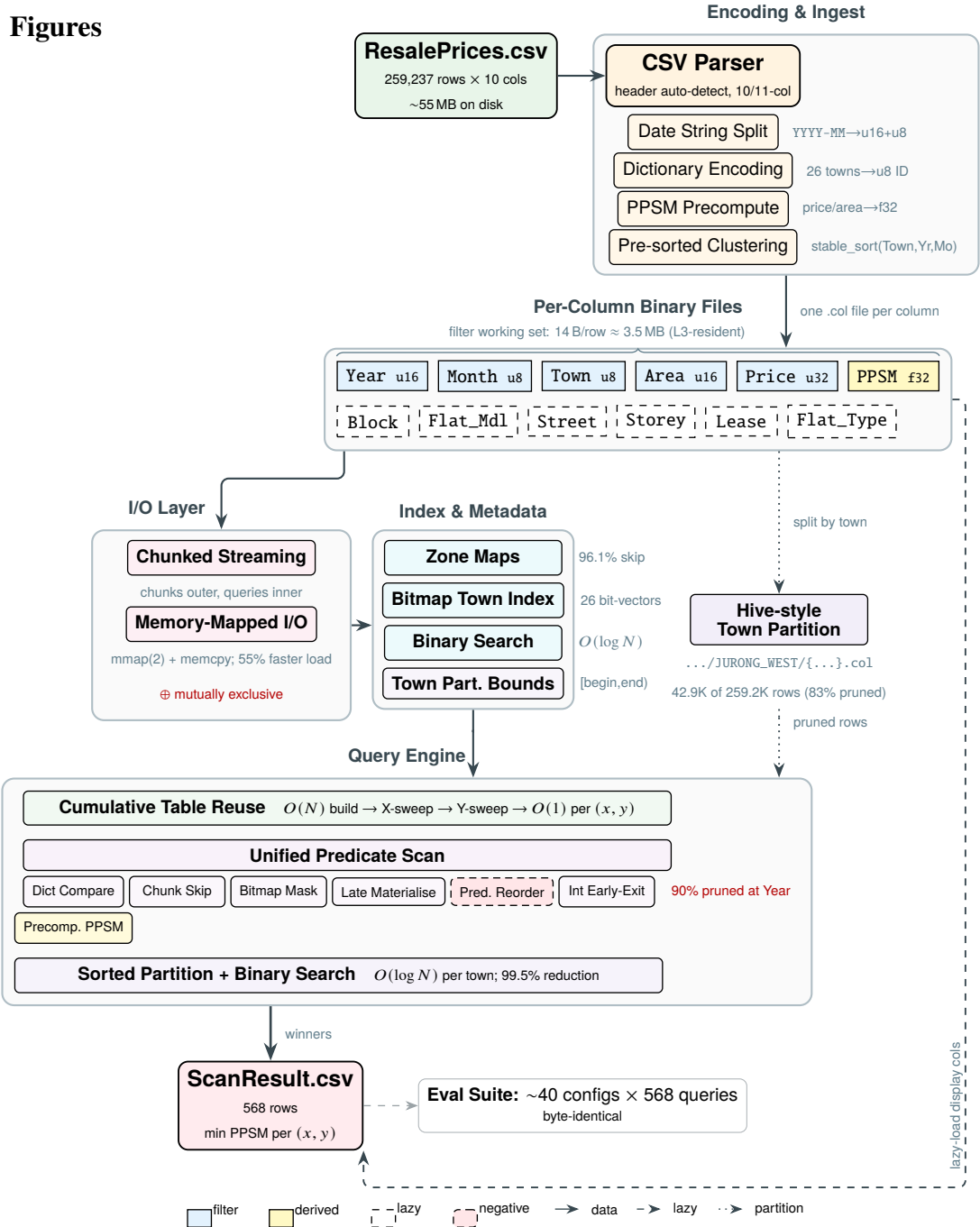


Figure 4: Data Flow and Processing Architecture

SC4023 Excel Cross-Validation — Matric U2220136J									
Year=2016 StartMonth=3		Towns: CLEMENTI, BEDOK, BUKIT PANJANG, CHOA CHU KANG, PASIR RIS							
(x, y)	Excel_PPSM	Valid?	Year	Month	Town	Block	Floor_Area	Flat_Model	Lease_Commence_Date
(1, 80)	3028	YES	2016	3	CHOA CHU KANG	701	109	Model A	1995
(3, 90)	2878	YES	2016	5	CHOA CHU KANG	464	123	Premium Apartment	2000
(4, 100)	2788	YES	2016	6	BUKIT PANJANG	213	104	Model A	1988
(6, 120)	2878	YES	2016	5	CHOA CHU KANG	464	123	Premium Apartment	2000
(7, 130)	2939	YES	2016	7	CHOA CHU KANG	540	132	Model A	1994
(8, 150)	3387	YES	2016	6	PASIR RIS	219	150	Apartment	1993

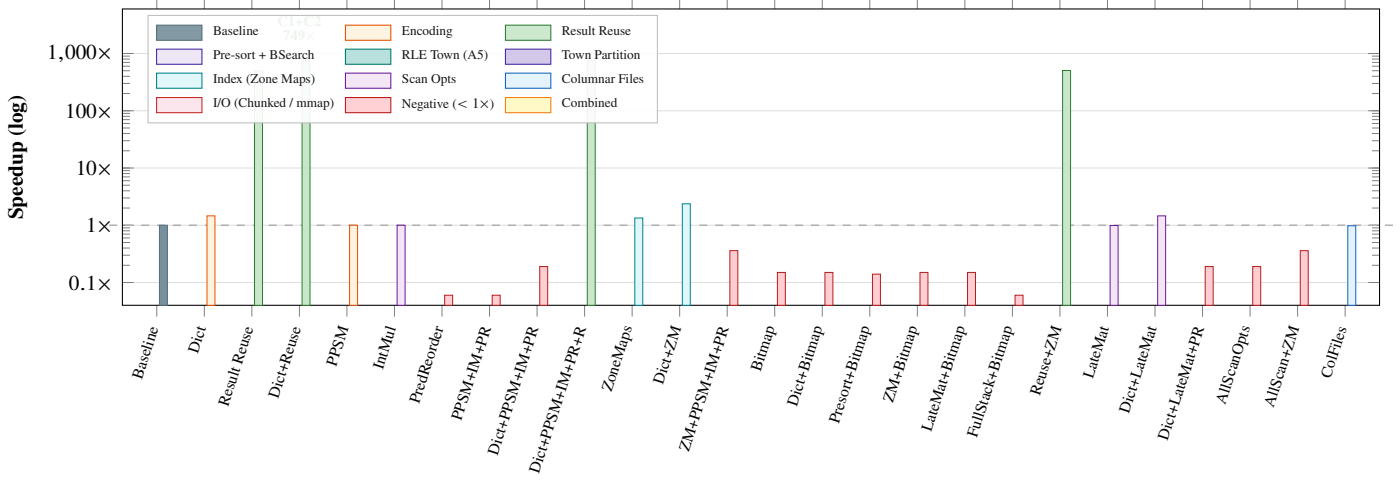
(a) Office Script editor and Validation sheet output.

```
ScanResult_U2220136J.csv ScanResult_U2220136J validation.csv Makefile README.md
Project > ScanResult_U2220136J validation.csv
7 (x, y),Year,Month,Town,Block,Floor_Area,Flat_Model,Lease_Commence_Date,Price_Per_Square_Meter
6 (1, 80),2016,03,CHOA CHU KANG,701,109,Model A,1995,3028
5 (3, 90),2016,05,CHOA CHU KANG,464,123,Premium Apartment,2000,2878
4 (4, 100),2016,06,BUKIT PANJANG,213,104,Model A,1988,2788
3 (6, 120),2016,05,CHOA CHU KANG,464,123,Premium Apartment,2000,2878
2 (7, 130),2016,07,CHOA CHU KANG,540,132,Model A,1994,2939
1 (8, 150),2016,06,PASIR RIS,219,150,Apartment,1993,3387
8
```

(b) Matching rows in ScanResult_U2220136J.csv.

Figure 5: Excel cross-validation: Office Script output (above) vs. engine CSV output (below). PPSM values match exactly for all tested pairs.

Encoding, Scan, Indexing, Columnar



I/O, Partitioning, Pre-sort, and Combined Configurations

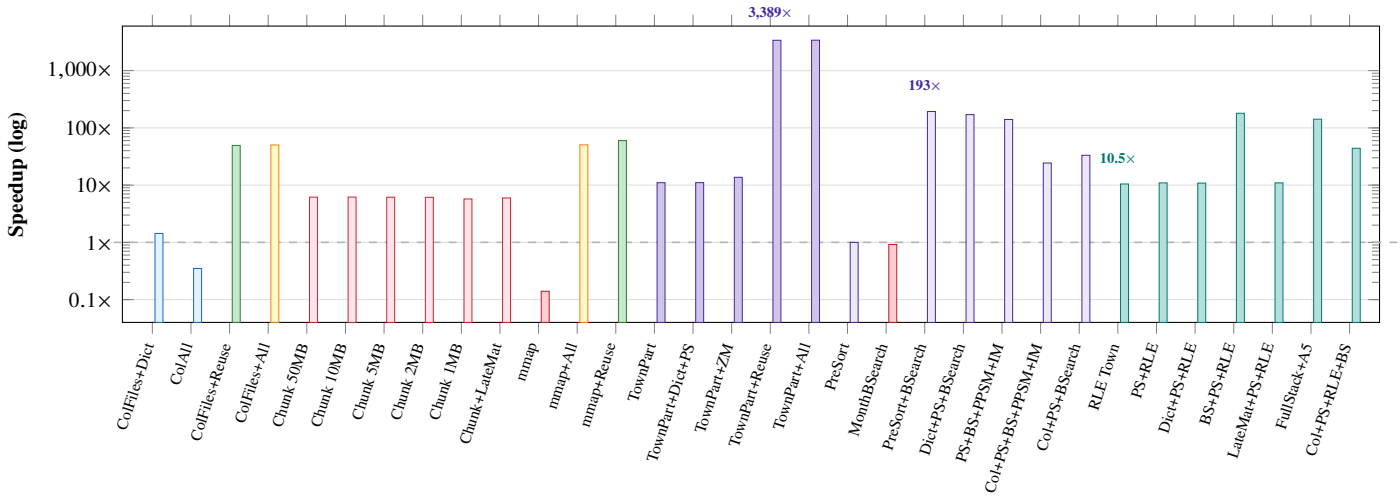


Figure 6: Speedup vs. baseline across all configurations (log scale, two rows).

```

1 std::vector<std::vector<MinEntry>> buildCumulativeTable(
2     const ColumnStore& db, uint16_t target_year,
3     uint8_t start_month, const std::vector<std::string>& towns) {
4     const std::size_t N = db.size();
5     std::vector<std::vector<MinEntry>> per_x(9, std::vector<MinEntry>(151));
6     // Pre-resolve town IDs (dict-encoded) or build hash set (string path)
7     std::unordered_set<std::string> town_set;
8     std::vector<uint16_t> town_ids;
9     if (db.use_dict_encoding)
10         for (auto& t : towns) { uint16_t id; if (db.dict_town.lookup(t,id))
11             town_ids.push_back(id); }
12     else town_set.insert(towns.begin(), towns.end());
13
14     // --- Step 1: Single O(N) scan --- per-(offset, bucket) minimum ---
15     for (std::size_t i = 0; i < N; ++i) {
16         if (db.col_month_year[i] != target_year) continue; // year filter
17         const uint8_t month = db.col_month_month[i];
18         if (month < start_month) continue;
19         int offset = int(month) - int(start_month) + 1;
20         if (offset < 1 || offset > 8) continue; // month range
21         if (db.use_dict_encoding) { // town filter
22             bool match = false; uint16_t rid = db.col_town_encoded[i];
23             for (auto& tid : town_ids) if (rid == tid) { match=true; break; }
24             if (!match) continue;
25         } else { if (town_set.find(db.col_town[i]) == town_set.end()) continue; }
26
27         const unsigned area = db.col_floor_area[i];
28         if (area < 80) continue; // below all y
29         const unsigned bucket = (area > 150) ? 150u : area; // clamp
30         const double ppsm = db.use_precomputed_ppsm
31             ? db.col_price_per_sqm[i]
32             : double(db.col_resale_price[i]) / double(db.col_floor_area[i]);
33         MinEntry& e = per_x[offset][bucket];
34         if (!e.has || ppsm < e.ppsm) { e.has=true; e.ppsm=ppsm; e.idx=i; }
35     }
36
37     // --- Step 2: X-sweep --- cumulative min across month offsets ---
38     std::vector<std::vector<MinEntry>> cum_x(9, std::vector<MinEntry>(151));
39     for (int area = 80; area <= 150; ++area) {
40         MinEntry running;
41         for (int off = 1; off <= 8; ++off) {
42             if (per_x[off][area].has)
43                 if (!running.has || per_x[off][area].ppsm < running.ppsm)
44                     running = per_x[off][area];
45             cum_x[off][area] = running;
46         }
47     }
48
49     // --- Step 3: Y-sweep --- propagate min from higher to lower area ---
50     for (int off = 1; off <= 8; ++off)
51         for (int area = 149; area >= 80; --area) {
52             const MinEntry& hi = cum_x[off][area+1]; MinEntry& lo
53                 = cum_x[off][area];
54             if (hi.has && (!lo.has || hi.ppsm < lo.ppsm)) lo = hi;
55         }
56     return cum_x; // O(1) lookup: cum_x[x][y]
57 }

```

Complexity: Step 1 = $O(N)$ | Steps 2+3 = $O(1)$ | Query: $568 \times O(1)$ lookups

Figure 7: C++ implementation of buildCumulativeTable from query_engine.cpp.